

PWM 舵机实验指导书（学生版）

适用平台：九联星闪开发板，基于海思 WS63E 芯片。

文档更新于 2026-05-21。

实验内容

序号	名称	做什么	对应章节
1	预备	舵机 PWM：50Hz、脉宽 0.5 ~ 2.5ms 与角度关系	第一节
2	<code>**pwm_servo**</code>	<code>make_app.py</code> 建工程 → 覆盖给定源码 → 编译烧录 → 观察 0°→180°→0° 扫描	第二 ~ 七节
3	拓展（必做·自编）	独立实现 <code>ServoSetAngle()</code> ，按指定角度序列转动	第八节

目的：理解舵机控制 PWM 原理，掌握 GPIO 软件 PWM 输出方法，并完成基础扫描实验；拓展部分练习角度到脉宽的换算与函数封装。

器材：PC、九联星闪开发板、USB 线、舵机（三线）、杜邦线。

一、舵机 PWM 原理要点

1.1 舵机如何转角度

- 舵机看的是 50Hz 方波（周期 20ms）里高电平有多宽。
- 0.5ms 高电平 ≈ 0°；1.5ms ≈ 90°；2.5ms ≈ 180°。

1.2 本实验参数

参数	取值
PWM 频率	50 Hz（周期 20 ms）
角度步进	18°（0°、18°、...、180°，共 11 档）
脉宽步进	档位 n=0 ~ 10，脉宽 = 500 + n×200（μs）
每档停留	300 ms

脉宽(us) = 500 + n × 200

角度(°) = n × 18

(n = 0 ... 10)

image-20260522113110170

1.3 为何用软件 PWM

WS63 硬件 PWM 最低约 1230Hz，做不出舵机需要的 50Hz。本实验在 GPIO10 上拉高/拉低，用 `uapi_tcxo_delay_us()` 控制脉宽（详见教师版与 [10-PWM的使用.md](#)）。

二、硬件连接

舵机线色	接到开发板
黄色（信号）	GPIO10
红色（电源）	3.3V
棕色（地）	GND

开发板 3.3V ———— 舵机红线（VCC）
开发板 GND ———— 舵机棕线（GND）
开发板 GPIO10 ——— 舵机黄线（信号）

注意：信号与电源必须**共地**。力矩不够时，舵机可用 5V 独立供电，信号线和 GND 仍接开发板；上电前确认极性，避免短路。

三、创建 pwm_servo 工程

在 OpenHarmony 源码**根目录**执行：

```
python3 applications/make_app.py -c pwm_servo
```

若目录已存在：

```
python3 applications/make_app.py -e pwm_servo
```

说明：脚本会在 `applications/sample/wifi-iot/app/pwm_servo/` 生成模板，并修改 `BUILD.gn`、`config.py`、`ohos.cmake`。目标名一般为 `**pwm_servo_demo**`。

推荐流程：先执行 `make_app.py`，再用本文 **第四节** 源码覆盖模板（与教师版 **第六节** 相同）。

四、pwm_servo 源码（请覆盖到工程目录）

路径： `applications/sample/wifi-iot/app/pwm_servo/`

与仓库 `applications/sample/wifi-iot/app/pwm_servo/pwm_servo.c`、教师版 **第六节** 全文一致，可直接复制。

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
```

```

* you may not use this file except in compliance with the License.
* You may obtain a copy of the License at
*
*     http://www.apache.org/licenses/LICENSE-2.0
*
* Unless required by applicable law or agreed to in writing, software
* distributed under the License is distributed on an "AS IS" BASIS,
* WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
* See the License for the specific language governing permissions and
* limitations under the License.
*/

/*
* 【基础实验】舵机扫描：GPIO10 输出 50Hz 软件 PWM
*
* 背景：
*   - 舵机要求周期 20ms（50Hz），高电平脉宽 0.5~2.5ms 表示角度。
*   - WS63 硬件 PWM 最低约 1230Hz，无法直接输出 50Hz。
*   - 本程序用 GPIO 手动拉高/拉低，配合 uapi_tcxo_delay_us() 构造脉宽。
*
* 行为：
*   - 档位 n=0..10，对应角度 0°、18°、...、180°（每档 +18°）。
*   - 每档停留 300ms 后进入下一档；正向扫完再反向回到 0°。
*/

#include <stdio.h>

#include "ohos_init.h"
#include "cmsis_os2.h"
#include "iot_gpio.h"
#include "iot_watchdog.h"
#include "tcxo.h"

/* ===== 引脚与任务配置 ===== */

/* 舵机信号线（黄线）接排针 GPIO10 */
#define SERVO_GPIO 10
/* 舵机任务栈大小（字节） */
#define SERVO_TASK_STACK_SIZE 0x1000
/* 任务优先级，与 gpio_led 等示例同级 */
#define SERVO_TASK_PRIO 25

/* ===== 舵机 PWM 时序（50Hz） ===== */

/*
* 一个 PWM 周期的总时长（微秒）。
* 50Hz → T = 1/50 s = 20ms = 20000us
*/
#define SERVO_PWM_PERIOD_US 20000

/* 舵机标准脉宽端点（单位：微秒） */
#define SERVO_PULSE_MIN_US 500 /* 0° 对应 0.5ms 高电平 */
#define SERVO_PULSE_MID_US 1500 /* 90° 对应 1.5ms（用于启动打印对照） */
#define SERVO_PULSE_MAX_US 2500 /* 180° 对应 2.5ms */

/*

```

```

* 基础实验用「档位」而非连续角度：
*   脉宽(us) = SERVO_PULSE_MIN_US + n * SERVO_PULSE_STEP_US
*   n=0 → 500us; n=10 → 2500us; 步进 200us
*/
#define SERVO_PULSE_STEP_US      200
#define SERVO_DUTY_STEP          10      /* 档位 0~10, 共 11 个位置 */

/* 档位与角度的对应：角度(°) = n * 18 */
#define SERVO_ANGLE_STEP_DEG     18

/* 每个档位保持时间（毫秒），期间重复输出多个 20ms 周期 */
#define SERVO_HOLD_MS            300

/*
* LiteOS 系统 tick 频率为 100Hz，即 1 tick = 10ms。
* osDelay(n) 的参数是 tick 数，不是毫秒；500ms 应写 MS_TO_TICKS(500)=50。
*/
#define TICK_PER_SEC              100
#define MS_TO_TICKS(ms)          (((ms) * TICK_PER_SEC) / 1000)

/* ===== 底层：单周期与档位保持 ===== */

/*
* ServoSendPulseUs - 输出一个完整的 50Hz PWM 周期
*
* @param pulse_us 本周期高电平宽度（0.5ms~2.5ms 范围内有效）
*
* 时序：先拉高 pulse_us → 再拉低 (20ms - pulse_us) → 周期结束。
* 舵机根据「高电平有多宽」判断目标角度，低电平段仅为填充周期。
*/
static void ServoSendPulseUs(uint32_t pulse_us)
{
    /* 防止脉宽超过整周期导致 delay 下溢 */
    if (pulse_us > SERVO_PWM_PERIOD_US) {
        pulse_us = SERVO_PWM_PERIOD_US;
    }

    /* 有效脉冲：逻辑高（3.3V） */
    IoTGpioSetOutputVal(SERVO_GPIO, IOT_GPIO_VALUE1);
    (void)uapi_tcxo_delay_us(pulse_us);

    /* 周期剩余时间保持低电平 */
    IoTGpioSetOutputVal(SERVO_GPIO, IOT_GPIO_VALUE0);
    (void)uapi_tcxo_delay_us(SERVO_PWM_PERIOD_US - pulse_us);
}

/*
* ServoDutyIndexToPulseUs - 档位 → 脉宽
*
* @param duty_index 档位 n，范围 0~SERVO_DUTY_STEP(10)
* @return          高电平宽度（us）
*/
static uint32_t ServoDutyIndexToPulseUs(unsigned int duty_index)
{
    return SERVO_PULSE_MIN_US + duty_index * SERVO_PULSE_STEP_US;
}

```

```

/*
 * ServoDutyIndexToAngle - 档位 → 显示用角度（仅用于 printf）
 */
static unsigned int ServoDutyIndexToAngle(unsigned int duty_index)
{
    return duty_index * SERVO_ANGLE_STEP_DEG;
}

/*
 * ServoHoldAtDutyIndex - 在某一档位保持 SERVO_HOLD_MS
 *
 * 只发 1 个周期舵机往来不及转动；需连续发多个相同脉宽的周期。
 * 300ms / 20ms ≈ 15 个周期。
 */
static void ServoHoldAtDutyIndex(unsigned int duty_index)
{
    uint32_t pulse_us = ServoDutyIndexToPulseUs(duty_index);
    unsigned int cycles = (SERVO_HOLD_MS * 1000U) / SERVO_PWM_PERIOD_US;

    for (unsigned int i = 0; i < cycles; i++) {
        ServoSendPulseUs(pulse_us);
    }
}

/* ===== 扫描: 0°→180° ===== */

/*
 * ServoSweep - 按档位扫描舵机
 *
 * @param forward 1: 档位 0→10（角度 0°→180°）
 *                0: 档位 10→0（角度 180°→0°）
 */
static void ServoSweep(unsigned int forward)
{
    if (forward) {
        for (unsigned int d = 0; d <= SERVO_DUTY_STEP; d++) {
            ServoHoldAtDutyIndex(d);
            printf("[pwm_servo] angle=%u deg, pulse=%u us\r\n",
                ServoDutyIndexToAngle(d), ServoDutyIndexToPulseUs(d));
            IoWatchDogKick(); /* 扫描较长，防止看门狗复位 */
        }
    } else {
        /* 从档位 10 递减到 1，最后单独回到档位 0 */
        for (unsigned int d = SERVO_DUTY_STEP; d > 0; d--) {
            ServoHoldAtDutyIndex(d);
            printf("[pwm_servo] angle=%u deg, pulse=%u us\r\n",
                ServoDutyIndexToAngle(d), ServoDutyIndexToPulseUs(d));
            IoWatchDogKick();
        }
        ServoHoldAtDutyIndex(0);
        printf("[pwm_servo] angle=0 deg, pulse=%u us\r\n",
            ServoDutyIndexToPulseUs(0));
        IoWatchDogKick();
    }
}

```

```

/* ===== 任务入口 ===== */

static void ServoTask(void *arg)
{
    (void)arg;

    /*
     * 实验要求第 1 步：初始化 GPIO10 为输出模式。
     * 软件 PWM 不经过硬件 PWM 外设，无需 IoSetFunc 复用。
     */
    IoTgpioInit(SERVO_GPIO);
    IoTgpioSetDir(SERVO_GPIO, IOT_GPIO_DIR_OUT);
    IoTgpioSetOutputVal(SERVO_GPIO, IOT_GPIO_VALUE0); /* 空闲低电平 */

    printf("[pwm_servo] 50Hz software PWM on GPIO%d, hold %dms/step\r\n",
           SERVO_GPIO, SERVO_HOLD_MS);
    printf("[pwm_servo] pulse: %uus(0deg) %uus(90deg) %uus(180deg)\r\n",
           SERVO_PULSE_MIN_US, SERVO_PULSE_MID_US, SERVO_PULSE_MAX_US);

    while (1) {
        /* 正向：0° → 180°，每步 18°，每步约 300ms */
        ServoSweep(1);
        osDelay(MS_TO_TICKS(500)); /* 两端扫描之间稍作停顿 */

        /* 反向：180° → 0° */
        ServoSweep(0);
        osDelay(MS_TO_TICKS(500));

        IoTWatchDogKick();
    }
}

/* 创建舵机控制任务 */
static void ServoEntry(void)
{
    osThreadAttr_t attr = {
        "ServoTask", 0, NULL, 0, NULL, SERVO_TASK_STACK_SIZE, SERVO_TASK_PRIO, 0,
0
    };

    if (osThreadNew(ServoTask, NULL, &attr) == NULL) {
        printf("[pwm_servo] Failed to create ServoTask!\r\n");
    }
}

APP_FEATURE_INIT(ServoEntry);

```

```

# Copyright (c) 2026 SZTU Open Source Organization.
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#

```

```
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

static_library("pwm_servo_demo") {
    sources = [ "pwm_servo.c" ]

    include_dirs = [
        "//commonlibrary/utils_lite/include",
        "//kernel/liteos_m/components/cmsis/2.0",
        "//base/iohardware/peripheral/interfaces/inner_api",
        "//device/soc/hisilicon/ws63v100/sdk/include/driver",
    ]
}
```

实现要点（详见源码注释）：

1. `IoTGpioInit` + `IoTGpioSetDir`：GPIO10 设为**输出**（软件 PWM，无需 `IoSetFunc`）。
2. `ServoSendPulseUs`：一个 20ms 周期内，先高电平 `pulse_us`，再低电平补足。
3. `ServoHoldAtDutyIndex` / `ServoSweep`：档位 0 ~ 10，角度步进 **18°**，每档 **300ms**。
4. 长循环中调用 `IoWatchDogKick()`；`osDelay` 须用 `MS_TO_TICKS`（100Hz tick）。

五、编译

```
python3 applications/make_app.py -b
```

或全量：

```
python3 applications/make_app.py -f
```

编译前确认 `applications/sample/wifi-iot/app/BUILD.gn` 的 `features` 中有 `"pwm_servo:pwm_servo_demo"`。

六、烧录

固件路径示例：

```
out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/ws63-liteos-app_all.fwpkg
```

使用 BurnTool：选择 **WS63**、对应串口，烧录后复位。详见 [03-环境配置和系统编译烧录文档.md](#)。

七、运行结果（`pwm_servo`）

与教师版 [第九节](#)、[11.3.5 完整串口示例](#) 一致。

7.1 舵机现象

- 每步 +18°，每步停留约 **300ms**（约 15 个 20ms PWM 周期）。
- **正向**：0° → 18° → ... → 180°（共 **11** 档）。
- **反向**：180° → ... → 0°；一轮结束后暂停约 **500ms**，再重复。

7.2 串口配置

参数	取值
波特率	115200 ，8N1
接口	开发板 USB 调试串口

7.3 启动行（上电后两条）

```
[pwm_servo] 50Hz software PWM on GPIO10, hold 300ms/step
[pwm_servo] pulse: 500us(0deg) 1500us(90deg) 2500us(180deg)
```

7.4 完整串口输出示例（一轮正向 + 一轮反向）

18° 间隔，正向 11 行 + 反向 11 行（与源码 `ServoSweep` 一致）：

```
[pwm_servo] 50Hz software PWM on GPIO10, hold 300ms/step
[pwm_servo] pulse: 500us(0deg) 1500us(90deg) 2500us(180deg)
[pwm_servo] angle=0 deg, pulse=500 us
[pwm_servo] angle=18 deg, pulse=700 us
[pwm_servo] angle=36 deg, pulse=900 us
[pwm_servo] angle=54 deg, pulse=1100 us
[pwm_servo] angle=72 deg, pulse=1300 us
[pwm_servo] angle=90 deg, pulse=1500 us
[pwm_servo] angle=108 deg, pulse=1700 us
[pwm_servo] angle=126 deg, pulse=1900 us
[pwm_servo] angle=144 deg, pulse=2100 us
[pwm_servo] angle=162 deg, pulse=2300 us
[pwm_servo] angle=180 deg, pulse=2500 us
[pwm_servo] angle=180 deg, pulse=2500 us
[pwm_servo] angle=162 deg, pulse=2300 us
[pwm_servo] angle=144 deg, pulse=2100 us
[pwm_servo] angle=126 deg, pulse=1900 us
[pwm_servo] angle=108 deg, pulse=1700 us
[pwm_servo] angle=90 deg, pulse=1500 us
[pwm_servo] angle=72 deg, pulse=1300 us
[pwm_servo] angle=54 deg, pulse=1100 us
[pwm_servo] angle=36 deg, pulse=900 us
[pwm_servo] angle=18 deg, pulse=700 us
[pwm_servo] angle=0 deg, pulse=500 us
```

说明：相邻角度行间隔约 **300ms**；脉宽公式 **500 + n×200**（n 为档位 0~10）。若串口为 `ext:`
`ServoSetAngle` 且仅 **45° 步进**，说明烧录了拓展程序，请改回本节 **第四节** 基础源码。

7.5 脉宽速查（验收抽查）

角度	脉宽 (us)
0°	500
90°	1500
180°	2500

八、拓展实验：按任意角度转动（必做·自编）

本节不提供完整参考答案。请在完成第四~七节、理解 `ServoSendPulseUs` 的基础上独立修改 `pwm_servo.c`。

8.1 实验目的

- 将「档位 n」改为「角度 (0~180°)」作为输入，练习线性换算脉宽。
- 练习函数封装，便于主程序按序列控制舵机。

8.2 功能要求（必须满足）

1. 新增函数 `**ServoSetAngle(unsigned int angle_deg)**`:
 - 参数范围 **0~180**（超出范围可钳位到 0 或 180，或返回错误，须在报告中说明）。
 - 脉宽按线性关系计算：`**pulse_us = 500 + angle_deg × 2000 / 180**`（与 0.5ms~2.5ms 对应）。
 - 在该角度下连续输出若干个完整 PWM 周期（建议 **15** 个，约 **300ms**，与第七节一致），再返回。
2. 修改 `**ServoTask` 主循环**（可去掉 `ServoSweep` 或保留但不再调用）：
 - 依次调用 `ServoSetAngle` 转到 **0° → 18° → 36° → ... → 180°**（步进 **18°**，共 **11** 个角度）。
 - 每个角度停留时间与基础实验一致（建议 **300ms**，即约 **15** 个 PWM 周期；也可在 `ServoSetAngle` 内用循环实现）。
 - 再反向回到 **0°**（**180° → 162° → ... → 0°**）。
 - 整段动作完成后 `osDelay` 约 **500ms**，再重复。
3. 每转到一个角度，串口打印一行：**角度与本次使用的 pulse_us**（格式自定，须能辨认）。
4. 保留 **GPIO 初始化、喂狗**；代码须有简要注释。

8.3 提示（非答案）

- 可参考 `ServoHoldAtDutyIndex`：先算 `pulse_us`，再循环调用 `ServoSendPulseUs`。
- 角度 90° 时，`pulse_us` 应约为 **1500**。
- 勿使用 `IoTPwmStart(..., 50)` 输出 50Hz（硬件不支持）。

8.4 验收标准

- 烧录后舵机按 **0°、18°、36°、...、180°** 扫描 (**18° 步进**，共 **11** 档)，每档约 **300ms**，再反向回到 0° (与第七节现象一致)。
- 串口角度行步进为 **18°**，脉宽随行递增 (如 0°→500us, 18°→700us, ..., 180°→2500us)；完整格式可参考 **7.4**。
- 须使用 `ServoSetAngle` 实现 (报告或注释中说明与 `ServoSweep` / 档位法的区别)；启动行或报告中须含**学号或姓名缩写**。
- 抽查：**90°** 时 `pulse` 约 **1500 us**；运行稳定，无明显看门狗反复复位。

8.5 提交内容

- 修改后的 `pwm_servo.c`、`BUILD.gn` (若有改动)。
- 实验报告：脉宽公式推导、与档位法的区别、遇到的问题。
- 舵机动作或串口打印截图 1 张。

九、课堂 checklist

步骤	内容	完成
1	按第二节接线 (GPIO10 / 3.3V / GND, 共地)	☐
2	<code>make_app.py -c pwm_servo</code> ，覆盖第四节基础源码	☐
3	编译、烧录	☐
4	舵机 0°→180°→0° 扫描 (18° 步进)；串口与 7.4 一致	☐
5	完成第八节拓展： <code>ServoSetAngle</code> ，18° 步进 0°~180°	☐
6	提交报告与截图	☐

十、参考文档

- [10-PWM的使用.md](#): PWM 基础
- [06-GPIO的使用.md](#): GPIO 输出
- [04-创建新项目的方法.md](#): 新建工程
- [03-环境配置和系统编译烧录文档.md](#): 编译烧录